

Kubernetes Basics

Practical Lab Guide

IEEE Session | 2 Hours

By Maya Mnaizel

Overview

This practical lab has two parts. In Part 1, you will deploy a simple nginx application on Kubernetes and interact with it using kubectl. In Part 2, you will deploy the Prometheus and Grafana monitoring stack and observe your running application in real time.

Prerequisites: kubectl installed, access to a Kubernetes cluster (minikube, kind, or a cloud cluster), and Helm installed for Part 2.

Part 1: Deploy Your First Application

What We Are Building

We will deploy nginx as a simple web server, expose it as a Service so it can be reached, scale it to multiple replicas, and observe how Kubernetes manages the pods.

Step-by-Step Instructions

Step 1: Create a Namespace

A namespace gives your resources a logical boundary inside the cluster.

```
kubectl create namespace ieee-demo

# Verify it was created
kubectl get namespaces
```

Step 2: Write the Deployment Manifest

Save the following as `nginx-deployment.yaml`. This tells Kubernetes to run 2 copies of `nginx`.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-demo
  namespace: ieee-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx-demo
  template:
    metadata:
      labels:
        app: nginx-demo
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

Step 3: Apply the Manifest

```
kubectl apply -f nginx-deployment.yaml

# Watch pods come up (press Ctrl+C to stop watching)
kubectl get pods -n ieee-demo -w
```

Step 4: Expose It With a Service

Save the following as `nginx-service.yaml`. This makes the deployment reachable inside the cluster.

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
  namespace: ieee-demo
spec:
  selector:
    app: nginx-demo
  ports:
    - port: 80
      targetPort: 80
  type: NodePort
```

```
kubectl apply -f nginx-service.yaml

# Get the NodePort assigned
kubectl get svc -n ieee-demo
```

Step 5: Scale Up and Observe

Scale to 4 replicas and watch Kubernetes create the new pods automatically.

```
kubectl scale deployment nginx-demo -n ieee-demo --replicas=4

# Watch pods scale up
kubectl get pods -n ieee-demo -w

# Describe the deployment to see events
kubectl describe deployment nginx-demo -n ieee-demo
```

Step 6: Trigger a Rollout

Update the nginx image version to trigger a rolling update and observe zero-downtime deployment.

```
kubectl set image deployment/nginx-demo nginx=nginx:1.25 -n ieee-demo

# Watch the rollout
kubectl rollout status deployment/nginx-demo -n ieee-demo

# Roll back if needed
kubectl rollout undo deployment/nginx-demo -n ieee-demo
```

Step 7: Check Logs

```
# List your pods
kubectl get pods -n ieee-demo

# View logs from one pod (replace with your actual pod name)
kubectl logs <pod-name> -n ieee-demo

# Stream logs live
kubectl logs -f <pod-name> -n ieee-demo
```

Checkpoint: At this point your app is running, exposed, scaled, and you have triggered a rollout. This is the core Kubernetes workflow.

Part 2: Add the Grafana Monitoring Stack

What We Are Building

We will deploy Prometheus to collect metrics from the cluster and Grafana to visualize them. By the end, you will see your nginx pods breathing in a live dashboard.

Step-by-Step Instructions

Step 1: Add the Helm Repositories

```
helm repo add prometheus-community  
https://prometheus-community.github.io/helm-charts  
helm repo add grafana https://grafana.github.io/helm-charts  
helm repo update
```

Step 2: Create a Monitoring Namespace

```
kubectl create namespace monitoring
```

Step 3: Deploy Prometheus

This installs Prometheus with sensible defaults using the community Helm chart.

```
helm install prometheus prometheus-community/prometheus \  
  --namespace monitoring \  
  --set alertmanager.enabled=false \  
  --set pushgateway.enabled=false  
  
# Wait for pods to be ready  
kubectl get pods -n monitoring -w
```

Step 4: Deploy Grafana

```
helm install grafana grafana/grafana \  
  --namespace monitoring \  
  --set dashboard.json=grafana_dashboard.json
```

```
--set adminPassword=ieee-demo-2024
```

```
# Get the Grafana pod name  
kubectl get pods -n monitoring
```

Step 5: Access the Grafana UI

Port-forward Grafana to your local machine so you can open it in a browser.

```
kubectl port-forward svc/grafana 3000:80 -n monitoring  
  
# Open in your browser  
# http://localhost:3000  
# Username: admin  
# Password: ieee-demo-2024
```

Step 6: Connect Prometheus as a Data Source

1	Open Configuration > Data Sources Click the gear icon in the left sidebar
2	Click Add data source Select Prometheus from the list
3	Set the URL http://prometheus-server.monitoring.svc.cluster.local:80
4	Click Save and Test You should see a green confirmation message

Step 7: Import a Kubernetes Dashboard

Grafana has a library of prebuilt dashboards. We will import the Kubernetes cluster overview dashboard.

1	Go to Dashboards > Import Click the + icon, then Import
2	Enter dashboard ID 3119 This is the Kubernetes cluster monitoring dashboard
3	Select your Prometheus data source From the dropdown at the bottom
4	Click Import The dashboard loads with live cluster metrics

Step 8: What to Look At

Walk through these panels on the dashboard with your audience:

- CPU usage per pod: find your nginx-demo pods and watch their usage
- Memory usage: see the memory footprint of each workload
- Pod restarts: this is your first signal that something is wrong
- Network I/O: observe traffic going in and out of the cluster

Live Demo Tip: Scale your nginx deployment up and down while the dashboard is open and watch the pod count and CPU metrics respond in real time. This makes the cluster feel alive for the audience.

Cleanup

Run these commands after the session to remove everything created during the lab.

```
# Remove the demo application
kubectl delete namespace ieee-demo

# Remove the monitoring stack
helm uninstall prometheus -n monitoring
helm uninstall grafana -n monitoring
kubectl delete namespace monitoring
```

Quick Reference: kubectl Commands

Command	What It Does
<code>kubectl get pods -n <ns></code>	List all pods in a namespace
<code>kubectl get pods -n <ns> -w</code>	Watch pods in real time
<code>kubectl describe pod <name> -n <ns></code>	Show full details and events for a pod
<code>kubectl logs <pod-name> -n <ns></code>	View pod logs
<code>kubectl logs -f <pod-name> -n <ns></code>	Stream pod logs live
<code>kubectl apply -f <file>.yaml</code>	Create or update resources from a file
<code>kubectl delete -f <file>.yaml</code>	Delete resources defined in a file
<code>kubectl scale deployment <name> --replicas=N</code>	Scale a deployment to N replicas
<code>kubectl rollout status deployment/<name></code>	Check rollout progress
<code>kubectl rollout undo deployment/<name></code>	Roll back to the previous version
<code>kubectl port-forward svc/<name> <local>:<remote></code>	Forward a service port to your machine

Good luck with the session!